

Training pipeline

How every model in this project was trained: data, optimisation, int8 export and the training variants.

Every learned model except the legacy TCN_MHA was trained by `src/train.py` with the procedure below. The per-family documents give each run's settings, learning curve and outcome.

1. Data and examples

- Data: RR series in `data/series/*.txt` split by subject (142 training, 16 validation; one-recording subjects go to validation). Training uses the training subjects only; validation monitors early stopping and is scored at the end. The 10 test subjects were never opened during training.
- Windows: the streaming engine runs over every series; each beat with a full 30-beat window gives one example (features of beat n , target $dRR[n+1]$). The first 30 windows of every series are dropped (cold RLS). 20,755,925 training and 1,421,225 validation windows.
- Normalisation: z-score statistics (RR, dRR, each feature, the target) are fitted on the training subjects only and stored in the run's `config.json`; a zero variance gives std 1.
- Target: $dRR[n+1] = RR[n+1] - RR[n]$. Residual models (all but linear) learn the z-scored correction $dRR - rls_pred$; the prediction adds `rls_pred` back.
- RAM: the (N, 30, 2) sequence tensor is never built. Each window is stored as its features, its target and the start index into the concatenated RR series; batches are gathered on the fly by `tf.data`, and an assert checks that this path equals the one used at inference (`utils.make_inputs`).

2. Optimisation (Keras models: `mlp`, `gru`, `micro_tcn`, `tcn_lite`)

setting	value
optimiser	AdamW, weight decay $1e-4$
loss	Huber, delta 1 (in z units: quadratic up to about one target standard deviation, linear beyond, so outlier beats weigh less than with MSE)
batch	512 windows; 64 steps per Keras call (<code>steps_per_execution</code>) to remove the per-step overhead that dominates small models
epoch	from round 2 on: 2,000,000 windows drawn uniformly at random WITH replacement (3,906 steps, about 10% of the data, a fresh subset every epoch). Round-1 runs (linear, <code>mlp</code> , <code>gbdt</code> , <code>micro_tcn</code> , <code>tcn_lite</code>): one full pass over all training windows
learning rate	$1e-3$ from scratch, $3e-4$ for fine-tunes and warm starts; <code>ReduceLROnPlateau</code> halves it when the validation loss stops improving (patience $\max(2, \text{patience} / 3)$ epochs), floor $1e-5$
stopping	<code>EarlyStopping</code> on the validation loss, patience 10 epochs (6 for fine-tunes), restoring the best epoch's weights; at most 100 epochs (30 for fine-tunes, 90 for <code>gru</code>)
validation monitor	a fixed random subset of 200,000 validation windows each epoch (round 2 on); the final metrics use all validation windows
reproducibility	seed 211 for Python, NumPy, Keras and the window draws; deterministic TensorFlow ops; the data manifests' md5 and the library versions are stored in <code>config.json</code>
hardware	one GPU (TensorFlow 2.19.1), one job at a time through the <code>jobs/</code> queue; per-step time 3 ms (<code>mlp</code>) to 25 ms (<code>gru</code>)

3. Non-neural models

- linear: ridge regression (penalty $1e-3$, intercept not penalised) solved in closed form from float64 normal equations accumulated in chunks of 1M windows; no epochs, 0.6 s.
- `gbdt`: scikit-learn `HistGradientBoostingRegressor` on CPU, early stopping on the validation windows (see `04_gbdt`).

4. After training (every run)

- The saved model is reloaded from disk and scored in ms on all validation windows, next to `rls_ar` (and the parent for fine-tunes); everything goes to `runs/<tag>/config.json`, the per-epoch curve to `history.csv`.
- int8 export (Keras): full-integer post-training quantisation with TFLite (int8 weights, activations, input and output). Since 2026-10-08 the calibration is best-of-K: 8 fixed draws (2,000 and 5,000 training windows x 4 seeds) plus a plain 20,000-window reference are exported, and the draw with the smallest int8-vs-float gap on held-out TRAINING windows is kept: 20 training subjects excluded from every draw, 20,000 ordinary + 20,000 closed-loop windows filled by the run's own float model. An assert requires best-of-K to be no worse than the reference. Validation never takes part in the choice.

- Calibration history: round-1 runs were exported by the first train.py; from 2026-10-07 18:14 a bug calibrated on only 500 windows (int8 output saturated, gaps 3.0-3.7%), fixed the same evening with 20,000 windows and train.py --reexport_int8; best-of-K replaced it on 2026-10-08 for the round-3/4 finalists. Each run's document says which export it has.

5. Training variants used in later rounds

- **Fine-tune** (--init_from): continue from a trained run's weights with lr 3e-4, at most 30 epochs, patience 6 (tcn_lite_ft).
- **Warm start with extra inputs** (--init_from + --features): the new inputs get zero weights and their own training-fitted normalisation, so training starts from the parent's exact function (_rsa_ws runs).
- **Closed-loop data, DAD** (--dad_filler, --dad_share 0.5): the training series are cut into 5000-beat segments, 10-80% of the beats are dropped and filled by the parent model as on the pacemaker, the engine recomputes the features on the filled series and the target stays the TRUE next beat (rr_true[n+1] - rr_filled[n]); half of the batches' windows come from these segments (_dad runs). A first attempt only reached the first 429k windows of each half (a tf.data index-range bug) and was deleted; a check now stops training if the draws cannot reach every window.

Documents in this folder (each in English _en and Spanish _es)

file	content
00_training_pipeline	this document: the shared procedure
01_baselines	persistence, ema_mean, rls_ar: not trained; how rls_ar adapts online
02_linear	linear
03_mlp	mlp, mlp_nobp, mlp_rsa
04_gbd	gbd, gbd_all
05_gru	gru
06_micro_tcn	micro_tcn, micro_tcn_f8, micro_tcn_dad, micro_tcn_rsa, micro_tcn_rsa_ws
07_tcn_lite	tcn_lite, tcn_lite_ft (selected), tcn_lite_dad, tcn_lite_rsa, tcn_lite_rsa_ws
08_tcn_mha	the legacy TCN_MHA (trained outside this project)

Abbreviations and acronyms used in this document

term	meaning
AdamW	Adam optimiser with decoupled weight decay (the training algorithm of the neural models)
ar	feature-set name: autoregressive inputs (rr_dev and the 8 most recent dRR) of the linear model
CPU	central processing unit
DAD	data as demonstrator: training on inputs that contain the model's own closed-loop fills, with the true target (a remedy for exposure bias)
_dad	run-name suffix: fine-tuned with DAD (closed-loop-filled) training data
dRR	beat-to-beat change of the RR interval: $dRR[n+1] = RR[n+1] - RR[n]$ (ms); the quantity predicted
ema_mean	baseline predicting the next RR as an exponential moving average of the window
_f8	run-name suffix: 8 convolution filters instead of 16
float64	64-bit (double-precision) floating-point numbers
_ft	run-name suffix: fine-tuned (training continued from the parent's weights)
gbdt	gradient-boosted decision trees (run name)
GPU	graphics processing unit
gru	gated recurrent unit (run name)
HRV	heart rate variability
int8	8-bit integer: weights and activations quantised to 8-bit integers, as run on the microcontroller; _int8 = the int8 version of a run
lr	learning rate (step size of the optimiser)
MHA	multi-head attention (a transformer layer, not supported by the MCU toolchain)
mlp	multilayer perceptron (run name)
ms	milliseconds
MSE	mean squared error
nobp	run-name part: no band-pass (the fixed RSA band-pass features bp and bp_prev removed)
RAM	random-access memory
RLS	recursive least squares: an adaptive filter that updates its coefficients at every beat
rls_ar	baseline: adaptive per-patient AR(8) forecast by RLS
rls_pred	feature: RLS AR(8) forecast of the next dRR (ms), clamped to +/-300
RR	RR interval: time between two consecutive heartbeats (R waves of the ECG), in ms
_rsa	run-name suffix: uses the RSA-tracker features
TCN	temporal convolutional network (causal, dilated 1-D convolutions)
tcn	temporal convolutional network (run-name part)
TFLite	TensorFlow Lite: the runtime and file format for models on embedded devices
_ws	run-name suffix: warm start (initialised from the parent's weights, new inputs set to zero)
z	z-score: $(\text{value} - \text{mean}) / \text{standard deviation}$, with statistics from the training subjects